

Introduction to Recommender Systems

Recommender systems are among the most commercially valuable applications of machine learning. While they receive moderate attention in academia, they are foundational to the success of major tech companies.

- **Examples:** Amazon (product suggestions), Netflix (movie picks), food delivery apps (restaurant ideas).
 - **Impact:** For many of these companies, a significant fraction of total sales/consumption is driven by automated recommendations.
-

The Problem Setup: Movie Ratings Example

To understand how these systems work, we use a running example of a movie streaming platform where users rate movies on a scale of **0 to 5 stars**.

The Data:

Imagine a dataset with 4 users and 5 movies.

Movie (Item)	Alice (1)	Bob (2)	Carol (3)	Dave (4)
Love at last	5	5	0	0
Romance forever	5	?	?	0
Cute puppies of love	?	4	0	?
Nonstop car chases	0	0	5	5
Swords vs. Karate	0	0	5	?

- **Observations:**
 - **Alice & Bob:** Prefer romance/soft topics (rated 4 or 5) and dislike action (rated 0).
 - **Carol & Dave:** Prefer action (rated 5) and dislike romance (rated 0).
 - **The Goal:** The system needs to look at the missing ratings (marked with ?) and predict what score the user *would* give if they watched it.
 - **The Action:** Recommend movies that are predicted to receive high ratings (e.g., 5 stars).
-

Formal Notation

To build an algorithm, we need standard notation to describe this data.

- n_u : The number of **users**. (In this example, $n_u = 4$).
- n_m : The number of **movies** (items). (In this example, $n_m = 5$).
- $r(i, j)$: A flag indicating if a rating exists.
 - $r(i, j) = 1$ if user j **has** rated movie i .
 - $r(i, j) = 0$ if user j **has not** rated movie i .
- $y^{(i,j)}$: The actual rating given.
 - This value only exists if $r(i, j) = 1$.
 - Example: $y^{(3,2)} = 4$ (User 2 rated Movie 3 a score of 4).

The Approach

The core task of the recommender system is to predict values for the empty cells where $r(i, j) = 0$.

In the upcoming lessons, we will develop an algorithm to solve this, starting with an assumption that we have specific **features** about the movies (e.g., knowing exactly how much "Action" or "Romance" is in a film) and then evolving to techniques that work even without that data.

Content-Based Filtering: Using Features

In this approach, we assume we *have* specific features for each item (movie) and we use these features to predict a user's preference.

The Setup

- n_u : Number of users.
- n_m : Number of movies (items).
- n : Number of features per movie.
 - Example: $n = 2$, where x_1 = Romance score, x_2 = Action score.

We treat this like a **Linear Regression** problem, but with a twist: we fit a separate linear regression model for **each individual user**.

The Model

For every user j , we learn a set of parameters $w^{(j)}$ and $b^{(j)}$.

To predict the rating user j would give to movie i :

$$\text{Prediction} = w^{(j)} \cdot x^{(i)} + b^{(j)}$$

- $x^{(i)}$: The feature vector for movie i (e.g., [0.99, 0] for a romance movie).
- $w^{(j)}$: The user's "preference" vector (e.g., [5, 0] if they love romance and hate action).
- $b^{(j)}$: The user's bias term.

The Cost Function (For One User j)

We want to minimize the squared error between the predicted rating and the actual rating, summing only over movies the user has actually rated ($r(i, j) = 1$).

$$J(w^{(j)}, b^{(j)}) = \frac{1}{2} \sum_{i:r(i,j)=1} ((w^{(j)} \cdot x^{(i)} + b^{(j)}) - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{k=1}^n (w_k^{(j)})^2$$

- The first term is the **Mean Squared Error** (prediction vs. actual).
 - The second term is the **Regularization** term (to prevent overfitting).
 - Note: The normalization constant $1/m^{(j)}$ is removed for mathematical convenience, as it doesn't change the optimal parameters.
-

The Full Cost Function (For All Users)

To learn the preferences for *all* users simultaneously, we sum the cost function over all users ($j = 1$ to n_u).

$$J(\text{all } w, \text{all } b) = \frac{1}{2} \sum_{j=1}^{n_u} \sum_{i:r(i,j)=1} ((w^{(j)} \cdot x^{(i)} + b^{(j)}) - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2$$

By minimizing this total cost function using **Gradient Descent**, we can learn the optimal parameters (w, b) for every single user, allowing us to predict their ratings for movies they haven't seen yet.

The Problem

This method works great **IF** we have the features $x^{(i)}$ (Romance score, Action score, etc.). But in the real world, we rarely have this detailed data for every single movie.

Collaborative Filtering: Learning the Features

In the previous "Content-Based" approach, we assumed we had the features for every movie (e.g., x_1 = Romance, x_2 = Action). But in reality, we rarely have these specific attributes for every item.

Collaborative Filtering allows us to learn these features (x) from the data itself, simultaneously with the user preferences (w and b).

1. The Intuition: Reverse Engineering

Imagine we don't know the genre of "Movie 1," but we do know the preferences of our users.

- **Alice** ($w^{(1)}$): Loves Romance, Hates Action.
- **Bob** ($w^{(2)}$): Loves Romance, Hates Action.
- **Charlie** ($w^{(3)}$): Hates Romance, Loves Action.

If Alice and Bob both rate "Movie 1" with **5 stars**, and Charlie rates it **0 stars**, the algorithm can infer that "Movie 1" must have a high "Romance" feature and a low "Action" feature.

- **Content-Based**: Given features x , learn user parameters w, b .
 - **Collaborative Filtering**: Given ratings y , learn **both** x AND w, b .
-

2. The Cost Function

To make this work, we combine the cost functions for users and movies into one massive objective.

A. Optimization for Features (x)

If we already had user parameters (w, b), we could find the best features ($x^{(i)}$) for a specific movie i by minimizing the error across all users who rated that movie:

$$J(x^{(i)}) = \frac{1}{2} \sum_{j:r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{k=1}^n (x_k^{(i)})^2$$

B. The Combined Cost Function

To learn everything at once, we sum the error over **all valid user-movie pairs** (i, j) where a rating exists.

The total cost function $J(w, b, x)$ is:

$$J(w, b, x) = \frac{1}{2} \sum_{(i,j):r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \text{Regularization}$$

Regularization Terms:

To prevent overfitting, we add regularization for w , b , and x :

$$+\frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2$$

3. The Algorithm: Gradient Descent

Now, x is no longer fixed input data; it is a **parameter** to be learned, just like w and b .

We minimize the cost function $J(w, b, x)$ using **Gradient Descent**.

We update all parameters iteratively:

1. **Update $w^{(j)}$** : Taking the derivative with respect to w .
2. **Update $b^{(j)}$** : Taking the derivative with respect to b .
3. **Update $x^{(i)}$** : Taking the derivative with respect to x .

By repeating these updates, the algorithm converges on a set of user preferences (w, b) and movie features (x) that accurately predict the existing ratings.

Why "Collaborative"?

This is called **Collaborative Filtering** because the users represent a "collaboration."

- When Alice and Bob rate movies, they help the system learn the feature vector x for those movies.
- Once the system learns the features x for a movie, it can use those features to predict how **Charlie** will rate it (based on Charlie's w).
- Essentially, Alice and Bob's ratings help predict Charlie's ratings, even if they have never met.

Collaborative Filtering for Binary Labels

In many real-world applications, user feedback isn't a 1-5 star rating. It's often binary: a simple "Yes/No," "Clicked/Didn't Click," or "Watched/Didn't Watch."

The Data

The labels in our dataset change from ratings (0-5) to:

- **1:** The user engaged (Liked, Purchased, Watched > 30s, Clicked).
- **0:** The user did *not* engage (Did not buy, skipped the video, ignored the ad) after being shown the item.
- **?** (Question Mark): The user has not yet seen the item.

The Transformation: From Linear to Logistic

Just as we moved from Linear Regression to Logistic Regression in supervised learning, we do the same here.

- **Old Prediction (Ratings):**

$$f(x) = w^{(j)} \cdot x^{(i)} + b^{(j)}$$

(This output a continuous number like 4.5).

- **New Prediction (Binary Probability):**

We wrap the linear equation in the **sigmoid (logistic) function** $g(z)$ to force the output between 0 and 1.

$$g(z) = \frac{1}{1+e^{-z}}$$

$$f(x) = g(w^{(j)} \cdot x^{(i)} + b^{(j)})$$

This output represents the **probability** that user j will engage with item i ($y = 1$).

The Cost Function (Binary Cross-Entropy)

Since we are now doing classification (0 or 1), the "Mean Squared Error" cost function is no longer appropriate. We switch to the **Binary Cross-Entropy Loss** (the standard loss for logistic regression).

Loss for a single example:

$$\text{Loss}(f(x), y) = -y \log(f(x)) - (1 - y) \log(1 - f(x))$$

Total Cost Function:

We sum this loss over all pairs (i, j) where a label exists ($r(i, j) = 1$).

$$J(w, b, x) = \sum_{(i,j):r(i,j)=1} \text{Loss}(f_{w,b,x}(x), y^{(i,j)}) + \text{Regularization}$$

By minimizing this cost function using gradient descent, the algorithm learns to predict the **probability** of a user liking an item.

Summary

- **Input:** Binary data (1 for engaged, 0 for not engaged).
- **Model:** Logistic function applied to the dot product of user/item vectors.
- **Optimization:** Minimize binary cross-entropy loss.
- **Result:** A system that predicts the *likelihood* of engagement, which is perfect for recommending items (e.g., "Show the user the items with the highest predicted probability of a click").